

C++ Overloading (Function and Operator)

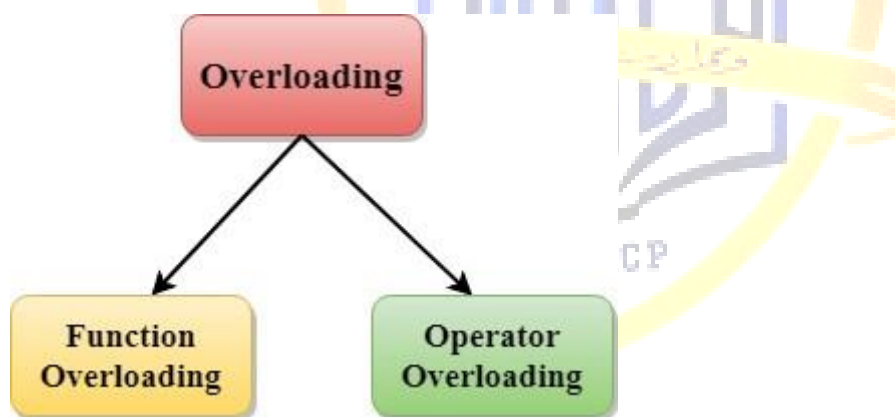
If we create two or more members having the same name but different in number or type of parameter, it is known as C++ overloading. In C++, we can overload:

- methods, ○
- constructors, and ○
- indexed properties

It is because these members have parameters only.

Types of overloading in C++ are:

- Function overloading
- Operator overloading



C++ Function Overloading

Function Overloading is defined as the process of having two or more function with the same name, but different in parameters is known as function overloading in C++. In function overloading, the function is redefined by using either different types of arguments or a different number of arguments. It is only through these differences compiler can differentiate between the functions.

The **advantage** of Function overloading is that it increases the readability of the program because you don't need to use different names for the same action.

C++ Function Overloading Example

Let's see the simple example of function overloading where we are changing number of arguments of add() method.

// program of function overloading when number of arguments vary.

```
1.  #include <iostream>
2.  using namespace std;
3.  class Cal {
4.  public:
5.  static int add(int a,int b){
6.  return a + b;
7.  }
8.  static int add(int a, int b, int c)
9.  {
10. return a + b + c;
11. }
12. };
13. int main(void) {
14. Cal C; // class object declaration.
15. cout<<C.add(10, 20)<<endl;
16. cout<<C.add(12, 20, 23);
17. return 0; 18. }
```

Output:

```
30
55
```

Let's see the simple example when the type of the arguments vary.

// Program of function overloading with different types of arguments.

```
1. #include<iostream>
2. using namespace std;
```

```

3.  int mul(int,int);  4. float mul(float,int);
5.
6.
7.  int mul(int a,int b)
8.  {
9.      return a*b;
10. }
11. float mul(double x, int y)
12. {
13.     return x*y;
14. }
15. int main()
16. {
17.     int r1 = mul(6,7);
18.     float r2 = mul(0.2,3);
19.     std::cout << "r1 is : " << r1 << std::endl;
20.     std::cout << "r2 is : " << r2 << std::endl;
21.     return 0;
22. }

```

Output:

```

r1 is : 42
r2 is : 0.6

```

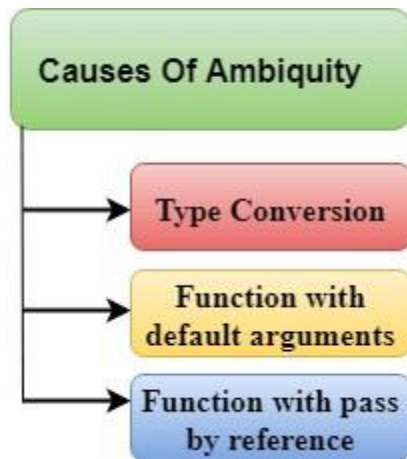
Function Overloading and Ambiguity

When the compiler is unable to decide which function is to be invoked among the overloaded function, this situation is known as **function overloading**.

When the compiler shows the ambiguity error, the compiler does not run the program.

Causes of Function Overloading:

- Type Conversion.
- Function with default arguments.
- Function with pass by reference.



- Type Conversion:

Let's see a simple example.

```
1. #include<iostream>
2. using namespace std;
3. void fun(int);
4. void fun(float);
5. void fun(int i)
6. {
7.     std::cout << "Value of i is : " << i << std::endl;
8. }
9. void fun(float j)
10. {
11.     std::cout << "Value of j is : " << j << std::endl;
12. }
13. int main()
14. {
15.     fun(12);
16.     fun(1.2);
17.     return 0; 18. }
```

The above example shows an error "**call of overloaded 'fun(double)' is ambiguous**". The fun(10) will call the first function. The fun(1.2) calls the second function according to

our prediction. But, this does not refer to any function as in C++, all the floating point constants are treated as double not as a float. If we replace float to double, the program works. Therefore, this is a type conversion from float to double.

- Function with Default Arguments

Let's see a simple example.

```
1.  #include<iostream>
2.  using namespace std;
3.  void fun(int);
4.  void fun(int,int);
5.  void fun(int i)
6.  {
7.  std::cout << "Value of i is : " << i << std::endl;
8.  }
9.  void fun(int a,int b=9)
10. {
11. std::cout << "Value of a is : " << a << std::endl;
12. std::cout << "Value of b is : " << b << std::endl;
13. }
14. int main()
15. {
16. fun(12);
17.
18. return 0;
19. }
```

The above example shows an error "call of overloaded 'fun(int)' is ambiguous". The fun(int a, int b=9) can be called in two ways: first is by calling the function with one argument, i.e., fun(12) and another way is calling the function with two arguments, i.e., fun(4,5). The fun(int i) function is invoked with one argument. Therefore, the compiler could not be able to select among fun(int i) and fun(int a,int b=9).

- Function with pass by reference

Let's see a simple example.

```
1. #include <iostream>
2. using namespace std;
3. void fun(int);
4. void fun(int &);
5. int main()
6. {
7.   int a=10;
8.   fun(a); // error, which f()?
9.   return 0;
10.}
11. void fun(int x)
12. {
13.   std::cout << "Value of x is : " <<x<< std::endl;
14. }
15. void fun(int &b)
16. {
17.   std::cout << "Value of b is : " <<b<< std::endl;
18. }
```

The above example shows an error "**call of overloaded 'fun(int&)' is ambiguous**". The first function takes one integer argument and the second function takes a reference parameter as an argument. In this case, the compiler does not know which function is needed by the user as there is no syntactical difference between the fun(int) and fun(int &).

C++ Operators Overloading

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type. For example, C++ provides

the ability to add the variables of the user-defined data type that is applied to the built-in data types.

The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

- Scope operator (::) ◦ Sizeof ◦
- member selector(.) ◦ member
- pointer selector(*) ◦ ternary
- operator(?:)

Syntax of Operator Overloading

```
1. return_type class_name :: operator op(argument_list)
2. {
3.     // body of the function.
4. }
```

Where the **return type** is the type of value returned by the function. **class_name** is the name of the class.

operator op is an operator function where op is the operator being overloaded, and the operator is the keyword.

Rules for Operator Overloading

- Existing operators can only be overloaded, but the new operators cannot be overloaded.
- The overloaded operator contains atleast one operand of the user-defined data type.
- We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.

- When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.
- When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

C++ Operators Overloading Example

Let's see the simple example of operator overloading in C++. In this example, void operator ++ () operator function is defined (inside Test class).

// program to overload the unary operator ++.



```
1.  #include <iostream>
2.  using namespace std;
3.  class Test
4.  {
5.  private:
6.  int num;
7.  public:
8.  Test(): num(8){}
9.  void operator ++()    {
10.     num = num+2;
11. }
12. void Print() {
13.     cout<<"The Count is: "<<num;
14. }
15. };
16. int main()
17. {
18.     Test tt;
19.     ++tt; // calling of a function "void operator ++()"
20.     tt.Print();
```



```
21.     return 0;
22. }
```

Output:

```
The Count is: 10
```

Let's see a simple example of overloading the binary operators.

// program to overload the binary operators.

```
1. #include <iostream>
2. using namespace std;
3. class A
4. {
5.
6.     int x;
7.     public:
8.     A(){}
9.     A(int i)
10.    {
11.        x=i;
12.    }
13.    void operator+(A);
14.    void display();
15. };
16.
17. void A :: operator+(A a)
18. {
19.
20.     int m = x+a.x;
21.     cout<<"The result of the addition of two objects is : "<<m;
22.
23. }
```



```
24.  int main()
25.  {
26.    A a1(5);
27.    A a2(4);
28.    a1+a2;
29.    return 0;
30. }
```

Output:

```
The result of the addition of two objects is : 9
```

C++ Function Overriding

If derived class defines same function as defined in its base class, it is known as function overriding in C++. It is used to achieve runtime polymorphism. It enables you to provide specific implementation of the function which is already provided by its base class.

C++ Function Overriding Example

Let's see a simple example of Function overriding in C++. In this example, we are overriding the eat() function.

```
1.  #include <iostream>
2.  using namespace std;
3.  class Animal {
4.  public:
5.  void eat(){
6.    cout<<"Eating...";
7.  }
8.  };
9.  class Dog: public Animal
10. {
11. public:
```

```
12. void eat()
13. {
14.     cout<<"Eating bread...";
15. }
16. };
17. int main(void) {
18.     Dog d = Dog();
19.     d.eat();
20.     return 0;
21. }
```

Output:

```
Eating bread...
```

